

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ПЕРСПЕКТИВНОЙ ИНЖЕНЕРИИ
ДЕПАРТАМЕНТ ЦИФРОВЫХ, РОБОТЕХНИЧЕСКИХ
СИСТЕМ И ЭЛЕКТРОНИКИ

КУРСОВАЯ РАБОТА
по дисциплине
«ИНТЕРНЕТ ТЕХНОЛОГИИ»
на тему:
«Разработка веб-приложения для аренды автомобилей.»

Выполнил:
Валенко Дмитрий Владимирович
Студент 4 курса группы ПИН-б-з-22-1
Направления подготовки
09.03.03 Прикладная информатика
заочной формы обучения

(подпись)
Руководитель работы:
Щеголев А.А.
(ФИО)

Работа допущена к защите _____
(подпись руководителя) (дата)

Работа выполнена и
защищена с оценкой _____ Дата защиты _____

Члены комиссии: _____
(должность) (подпись) (И.О. Фамилия)

Ставрополь, 2026 г

УТВЕРЖДАЮ

директор департамента цифровых,
робототехнических систем
и электроники
_____ Азаров И.В.

Институт Перспективной инженерии
Департамент Цифровых, робототехнических систем и
электроники
Направление подготовки 09.03.03
Направленность (профиль) Прикладная информатика

ЗАДАНИЕ

на курсовую работу/проект

студента Валенко Дмитрий Владимирович
(фамилия, имя, отчество)

по дисциплине Интернет-технологии

1. Тема работы «Разработка веб-приложения для аренды автомобилей»

2. Цель - разработать веб-приложение для аренды автомобилей, которое позволит пользователям бронировать автомобили, а также отслеживать историю своих бронирований.

3. Задачи Сбор и анализ информации: Поиск и изучение литературы, статей, научных исследований и других источников информации по выбранной теме. Планирование работы: Составление плана курсовой работы, определение основных этапов и сроков выполнения. Написание текста: Оформление результатов исследования в виде структурированного текста, включающего введение, основную часть и заключение. Проведение экспериментов и практическая реализация: Если это необходимо, выполнение практических заданий, связанных с разработкой программного обеспечения, созданием веб-сайтов или проведением экспериментов. Оформление работы: Соблюдение требований к оформлению курсовой работы, включая правильное оформление списка литературы, таблиц, рисунков и схем. Представление и защита работы: Подготовка презентации и выступление перед комиссией, где студент демонстрирует результаты своего труда и отвечает на вопросы.

4. Перечень подлежащих разработке вопросов:

а) по теоретической части Исторический обзор развития интернет-технологий (в зависимости от выбранной темы: стека технологий). Основные понятия и определения. Описание выбранного стека технологий. Анализ существующих решений и подходов.

б) по практической части Постановка задачи. Выбор методов и инструментов для решения задачи. Описание процесса разработки. Результаты эксперимента или практической реализации. Оценка эффективности предложенных решений.

5. Исходные данные:

а) по литературным источникам _____

б) по вариантам, разработанным преподавателем _____

в) иное _____

6. Список рекомендуемой литературы _____

7. Контрольные сроки представления отдельных разделов курсовой работы:

25 % - _____ “ ” _____ 20_ г.

50 % - _____ “ ” _____ 20_ г.

75 % - _____ “ ” _____ 20_ г.

100 % - _____ “ ” _____ 20_ г.

8. Срок защиты студентом курсовой работы “ ” _____ 20_ г.

Дата выдачи задания “ ” _____ 20_ г.

Руководитель курсовой работы

Старший преподаватель департамента цифровых, робототехнических систем
и электроники института перспективной инженерии Щеголев А.А.

(ученая степень, звание)(личная подпись)(инициалы, фамилия)

Задание принял к исполнению студент заочной формы обучения 4 курса
ПИН-6-3-22-1 группы _____

(личная подпись)

(инициалы, фамилия)

Оглавление

ВВЕДЕНИЕ	5
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЙ.....	8
1.1 Исторический обзор развития интернет-технологий	8
1.2 Основные понятия веб-разработки.....	10
1.3 Описание выбранного стека технологий.....	13
1.4 Анализ существующих решений для аренды автомобилей.....	16
2 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ АРЕНДЫ АВТОМОБИЛЕЙ...	22
2.1 Постановка задачи	22
2.2 Выбор средства создания веб-приложения	24
2.3 Описание объектной модели	27
2.4 База данных и серверная часть.....	31
2.5 Разработка клиентской части	35
2.6 Использование системы контроля версий, развертывание	38
2.7 Тестирование и верификация работоспособности	40
ЗАКЛЮЧЕНИЕ.....	43
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	46

ВВЕДЕНИЕ

Актуальность темы обусловлена активным развитием цифровых технологий и ростом популярности онлайн-сервисов в сфере транспортных услуг. В современных условиях пользователи стремятся получать доступ к необходимым сервисам быстро и удобно, не посещая офисы компаний лично. Одним из наиболее востребованных направлений является аренда автомобилей через интернет, позволяющая клиентам выбирать транспортные средства, оформлять бронирование и получать необходимую информацию в режиме реального времени. В связи с этим разработка веб-приложений для автоматизации процесса аренды автомобилей представляет значительный практический интерес и отвечает современным требованиям рынка.

Объект исследования – процесс разработки веб-приложений для автоматизации услуг аренды автомобилей с использованием современных веб-технологий.

Предмет исследования – методы, средства и технологии проектирования и реализации веб-приложения для аренды автомобилей, а также организация взаимодействия клиентской и серверной частей системы.

Цель работы – разработка и документирование веб-приложения для аренды автомобилей, обеспечивающего удобный поиск, просмотр и бронирование транспортных средств, а также демонстрирующего практическое применение современных технологий веб-разработки.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ современных подходов к разработке веб-приложений и изучить особенности организации онлайн-сервисов аренды автомобилей;
- исследовать существующие решения в области каршеринга и аренды транспортных средств, определить их основные преимущества и недостатки;
- сформулировать функциональные и нефункциональные требования к разрабатываемому приложению;

- обосновать выбор программных средств, языков программирования, библиотек и фреймворков для реализации проекта;
- разработать структуру базы данных для хранения информации об автомобилях, пользователях и бронированиях;
- реализовать серверную часть приложения, обеспечивающую обработку запросов пользователей и управление данными;
- разработать клиентскую часть приложения с удобным пользовательским интерфейсом для поиска и аренды автомобилей;
- реализовать механизм просмотра каталога автомобилей с возможностью фильтрации и получения подробной информации о транспортных средствах;
- реализовать функциональность оформления и управления бронированием автомобилей;
- реализовать функционал, позволяющий администратору управлять системой;
- организовать взаимодействие клиентской и серверной частей посредством API;
- использовать систему контроля версий Git для отслеживания изменений в проекте и размещения исходного кода в удаленном репозитории;
- провести тестирование разработанного приложения и оценить корректность его работы.

Методы исследования: анализ научной, учебной и технической литературы, изучение документации используемых технологий, сравнительный анализ существующих веб-сервисов аренды автомобилей, методы объектно-ориентированного проектирования, программирование и тестирование программного обеспечения.

Теоретическая значимость работы заключается в систематизации и обобщении знаний о современных подходах к разработке веб-приложений, архитектуре клиент-серверных систем, принципах построения

информационных сервисов и организации процессов бронирования транспортных средств в цифровой среде.

Практическая значимость работы состоит в создании работоспособного веб-приложения для аренды автомобилей, которое может использоваться в качестве основы для разработки коммерческого сервиса либо выступать учебным примером при изучении современных технологий веб-разработки. Полученные в ходе выполнения работы результаты могут быть расширены за счет внедрения дополнительных возможностей, таких как онлайн-оплата, система отзывов, управление пользовательскими аккаунтами, интеграция с картографическими сервисами и мобильными приложениями.

Структура работы соответствует логике исследования и включает введение, две главы, заключение, список использованных источников и приложения. В первой главе рассматриваются теоретические аспекты разработки веб-приложений, анализируются современные технологии и существующие сервисы аренды автомобилей, а также обосновывается выбор программных средств для реализации проекта. Во второй главе подробно описываются этапы практической разработки системы: постановка задачи, проектирование структуры приложения и базы данных, реализация серверной и клиентской частей, настройка взаимодействия компонентов системы, тестирование и анализ результатов. В заключении подводятся итоги выполненной работы, формулируются основные выводы и определяются перспективы дальнейшего развития разработанного веб-приложения.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЙ

1.1 Исторический обзор развития интернет-технологий

Развитие интернет-технологий является одним из важнейших факторов цифровой трансформации современного общества. Появление глобальной сети Интернет и последующее создание Всемирной паутины (World Wide Web) обеспечили возможность обмена информацией между пользователями независимо от их географического расположения. Основы современной веб-разработки были заложены в начале 1990-х годов, когда британский ученый Тим Бернерс-Ли предложил концепцию гипертекстовой системы, включающей язык разметки HTML, протокол передачи данных HTTP и систему адресации ресурсов URL.

На раннем этапе развития веб-технологий сайты представляли собой статические страницы, содержимое которых формировалось заранее и не изменялось в процессе взаимодействия с пользователем. Основной задачей таких ресурсов являлось предоставление текстовой информации и навигации по связанным документам. Несмотря на ограниченные возможности, данный подход стал основой для дальнейшего развития сети Интернет.

Следующим этапом стало появление технологий, позволяющих создавать динамические веб-страницы. Существенный вклад в развитие веб-разработки внес язык программирования JavaScript, представленный в 1995 году. Его использование позволило выполнять программный код непосредственно в браузере пользователя, обрабатывать события интерфейса и изменять содержимое страниц без повторной загрузки. В дальнейшем развитие технологий асинхронного обмена данными между клиентом и сервером обеспечило более высокий уровень интерактивности веб-приложений и повысило удобство работы пользователей.

Параллельно совершенствовались серверные технологии. Для обработки запросов пользователей начали использоваться специализированные языки программирования и программные платформы, обеспечивающие генерацию динамического содержимого и взаимодействие с

базами данных. Это позволило создавать более сложные информационные системы, включающие регистрацию пользователей, хранение данных, обработку заказов и другие бизнес-процессы.

Значительным этапом развития веб-технологий стало распространение концепции клиент-серверной архитектуры, при которой функции отображения информации и обработки данных разделяются между клиентской и серверной частями приложения. Такой подход повысил масштабируемость и удобство сопровождения программных систем. В дальнейшем широкое распространение получили программные интерфейсы приложений (API), обеспечивающие стандартизированное взаимодействие между различными компонентами информационных систем.

В начале XXI века развитие вычислительных мощностей и сетевой инфраструктуры привело к появлению современных веб-приложений, обладающих высокой производительностью и функциональностью. Веб-сайты постепенно превратились в полноценные программные системы, способные выполнять широкий спектр задач: электронную коммерцию, онлайн-бронирование услуг, управление корпоративными данными, дистанционное обучение и многие другие функции.

Отдельного внимания заслуживает развитие архитектурных подходов к построению программного обеспечения. Если ранее большинство приложений создавались в виде монолитных систем, то сегодня всё большую популярность приобретают микросервисные решения. Данный подход предполагает разделение приложения на независимые сервисы, каждый из которых отвечает за определенную бизнес-функцию. Это позволяет упростить поддержку программного обеспечения, повысить отказоустойчивость системы и обеспечить возможность независимого масштабирования отдельных компонентов.

Современный этап развития веб-разработки характеризуется широким использованием специализированных фреймворков, систем управления базами данных, технологий контейнеризации и средств автоматизации

процессов разработки и развертывания программного обеспечения. Для создания пользовательских интерфейсов применяются современные JavaScript-фреймворки и библиотеки, обеспечивающие высокую скорость работы и удобство взаимодействия с пользователем. Серверная часть приложений реализуется с использованием специализированных платформ и архитектурных решений, ориентированных на обеспечение надежности, безопасности и масштабируемости.

Таким образом, развитие интернет-технологий прошло путь от простых статических веб-страниц до сложных распределенных информационных систем, способных обслуживать миллионы пользователей одновременно. Современные технологии веб-разработки предоставляют широкие возможности для создания высокопроизводительных сервисов, в том числе систем онлайн-бронирования и аренды автомобилей, позволяющих автоматизировать бизнес-процессы и обеспечивать высокий уровень удобства для конечных пользователей.

1.2 Основные понятия веб-разработки

Для понимания принципов создания современных информационных систем необходимо рассмотреть основные понятия, используемые в области веб-разработки.

Веб-приложение представляет собой программную систему, функционирующую в сети Интернет и обеспечивающую взаимодействие пользователя с информационными ресурсами посредством веб-браузера. В отличие от традиционных настольных программ, веб-приложения не требуют установки на устройство пользователя и доступны с различных платформ при наличии доступа к сети.

Основой функционирования веб-приложений является клиент-серверная архитектура, предполагающая разделение системы на две основные части: клиентскую и серверную. Клиентская часть отвечает за отображение пользовательского интерфейса и взаимодействие с пользователем, а серверная

часть выполняет обработку запросов, хранение данных и реализацию бизнес-логики приложения.

Клиентская часть (Frontend) представляет собой программный код, выполняемый в браузере пользователя. Её основными задачами являются отображение информации, обеспечение удобного пользовательского интерфейса и обработка действий пользователя. Для разработки клиентской части традиционно используются следующие технологии:

- HTML (HyperText Markup Language) — язык гипертекстовой разметки, предназначенный для описания структуры веб-документов;
- CSS (Cascading Style Sheets) — язык таблиц стилей, используемый для оформления элементов интерфейса и адаптации внешнего вида веб-страниц к различным устройствам;
- JavaScript — язык программирования, обеспечивающий интерактивность веб-приложений и возможность динамического изменения содержимого страниц.

Серверная часть (Backend) представляет собой программное обеспечение, работающее на сервере и выполняющее обработку запросов клиентов. К её основным функциям относятся реализация бизнес-логики, обработка пользовательских данных, взаимодействие с базами данных, обеспечение безопасности и формирование ответов клиентской части.

Обмен информацией между клиентом и сервером осуществляется посредством HTTP (HyperText Transfer Protocol) — протокола прикладного уровня, предназначенного для передачи данных в сети Интернет. Каждый HTTP-запрос содержит адрес ресурса, метод обращения и дополнительную служебную информацию. В ответ сервер возвращает данные и специальный код состояния, характеризующий результат обработки запроса.

Наиболее распространёнными методами HTTP являются:

- GET — получение данных;
- POST — создание новых данных;
- PUT — полное обновление существующих данных;

- PATCH — частичное обновление данных;
- DELETE — удаление данных.

Для организации взаимодействия между различными программными компонентами используются программные интерфейсы приложений (API). API определяет набор правил и механизмов, позволяющих различным системам обмениваться информацией и выполнять совместную работу. В веб-разработке широкое распространение получил архитектурный подход REST, основанный на использовании стандартных возможностей протокола HTTP для выполнения операций над ресурсами.

Неотъемлемой частью большинства веб-приложений являются системы управления базами данных (СУБД), предназначенные для хранения и обработки информации. Базы данных позволяют организовать долговременное хранение данных, обеспечить их целостность и предоставить эффективные механизмы поиска и изменения информации. В современных информационных системах используются как реляционные, так и нереляционные модели хранения данных.

Для обеспечения безопасности веб-приложений применяются механизмы аутентификации и авторизации. Аутентификация предназначена для подтверждения личности пользователя, а авторизация определяет набор действий и ресурсов, доступных пользователю после успешного входа в систему. Реализация данных механизмов является важным условием защиты информации и предотвращения несанкционированного доступа к данным.

В процессе разработки программного обеспечения широко используются системы контроля версий, позволяющие отслеживать изменения исходного кода, сохранять историю разработки и организовывать совместную работу нескольких разработчиков над одним проектом. Применение подобных систем существенно повышает надёжность процесса разработки и упрощает сопровождение программного продукта.

Важную роль в современных веб-приложениях играет принцип разделения ответственности между отдельными компонентами системы.

Такой подход позволяет повысить масштабируемость, удобство сопровождения и качество программного обеспечения. Благодаря этому разработчики могут создавать сложные информационные системы, состоящие из множества взаимосвязанных модулей, каждый из которых выполняет строго определённый набор функций.

Таким образом, современная веб-разработка представляет собой совокупность технологий, архитектурных подходов и программных средств, обеспечивающих создание функциональных, надёжных и удобных информационных систем. Понимание основных понятий веб-разработки является необходимой основой для проектирования и реализации веб-приложения аренды автомобилей.

1.3 Описание выбранного стека технологий

Для реализации веб-приложения аренды автомобилей был выбран современный стек технологий, обеспечивающий высокую производительность, масштабируемость и удобство сопровождения программного продукта. При выборе инструментов учитывались требования к надёжности системы, возможности дальнейшего расширения функциональности, а также соответствие современным подходам к разработке программного обеспечения.

В качестве серверной платформы был выбран NestJS — прогрессивный фреймворк для разработки серверных приложений на языке TypeScript. Основным преимуществом NestJS является модульная архитектура, позволяющая разделять систему на независимые функциональные компоненты и обеспечивать высокий уровень структурированности программного кода. Фреймворк предоставляет встроенные механизмы внедрения зависимостей (Dependency Injection), поддержку объектно-ориентированного программирования и реализацию архитектурных паттернов, широко применяемых при разработке корпоративных информационных систем.

Использование языка TypeScript позволяет повысить надежность программного обеспечения за счет статической типизации, упрощает сопровождение проекта и снижает вероятность возникновения ошибок на этапе выполнения программы. Кроме того, TypeScript обеспечивает более высокий уровень читаемости и предсказуемости программного кода по сравнению с традиционным JavaScript.

Архитектура серверной части построена на основе принципов Domain-Driven Design (DDD) и SOLID. Такой подход позволяет разделить бизнес-логику и инфраструктурные компоненты, обеспечить слабую связанность модулей и повысить масштабируемость системы. Каждый сервис отвечает за выполнение строго определенного набора функций, что упрощает сопровождение и дальнейшее развитие программного продукта.

Для реализации системы выбрана микросервисная архитектура, предполагающая разделение приложения на несколько независимых сервисов. Каждый сервис отвечает за отдельную предметную область и может разрабатываться, тестироваться и развертываться независимо от остальных компонентов системы. Данный подход обеспечивает гибкость разработки, отказоустойчивость и возможность горизонтального масштабирования отдельных частей приложения при увеличении нагрузки.

Для хранения данных используется PostgreSQL — современная объектно-реляционная система управления базами данных. Выбор PostgreSQL обусловлен её высокой производительностью, поддержкой сложных запросов, механизмов транзакций и обеспечением целостности данных. Использование реляционной модели хранения особенно актуально для систем бронирования, где требуется поддержка взаимосвязанных сущностей, таких как пользователи, автомобили и заявки на аренду.

Взаимодействие серверной части с базой данных осуществляется посредством технологии ORM (Object-Relational Mapping). Данный подход позволяет работать с объектами предметной области средствами языка программирования без необходимости постоянного написания

низкоуровневых SQL-запросов. Использование ORM повышает производительность разработки и способствует поддержанию единого уровня абстракции при работе с данными.

Для обеспечения безопасности приложения реализованы механизмы аутентификации и авторизации пользователей на основе JSON Web Token (JWT). Данный подход позволяет безопасно идентифицировать пользователей системы и разграничивать права доступа к различным функциям приложения. Использование токенов обеспечивает независимость клиентской и серверной частей системы и широко применяется в современных веб-приложениях.

Клиентская часть системы разработана с использованием Vue.js — современного JavaScript-фреймворка для построения пользовательских интерфейсов. Основными преимуществами данного решения являются компонентный подход к разработке, высокая производительность и удобство сопровождения программного кода. Использование Vue позволяет создавать динамические пользовательские интерфейсы и обеспечивать комфортное взаимодействие пользователя с системой.

При проектировании клиентской части использовалась архитектура Feature-Sliced Design (FSD), ориентированная на разделение функциональности приложения по бизнес-сценариям и уровням ответственности. Такой подход повышает структурированность проекта, упрощает навигацию по исходному коду и способствует более эффективному сопровождению приложения по мере его развития.

Для организации взаимодействия между клиентом и сервером используется технология REST API, обеспечивающая передачу данных посредством HTTP-запросов. Данный подход является одним из наиболее распространённых способов интеграции компонентов современных веб-приложений благодаря своей простоте, универсальности и независимости от используемых технологий.

Развертывание программного комплекса осуществляется с использованием технологии Docker, обеспечивающей контейнеризацию

компонентов системы. Контейнеризация позволяет создавать единое окружение для разработки, тестирования и эксплуатации приложения, исключая различия между рабочими средами. Каждый сервис системы разворачивается в отдельном контейнере, что упрощает управление инфраструктурой и повышает переносимость программного продукта.

Для автоматизации процессов сборки, тестирования и развертывания используется технология CI/CD (Continuous Integration / Continuous Delivery). Автоматизация позволяет сократить количество ошибок, связанных с ручным развертыванием, ускорить выпуск новых версий программного обеспечения и обеспечить стабильность процесса разработки.

Контроль версий осуществляется посредством системы Git, обеспечивающей отслеживание изменений исходного кода и поддержку коллективной разработки. Для хранения удалённого репозитория используется платформа GitHub, предоставляющая инструменты для совместной работы, автоматизации процессов разработки и документирования проекта.

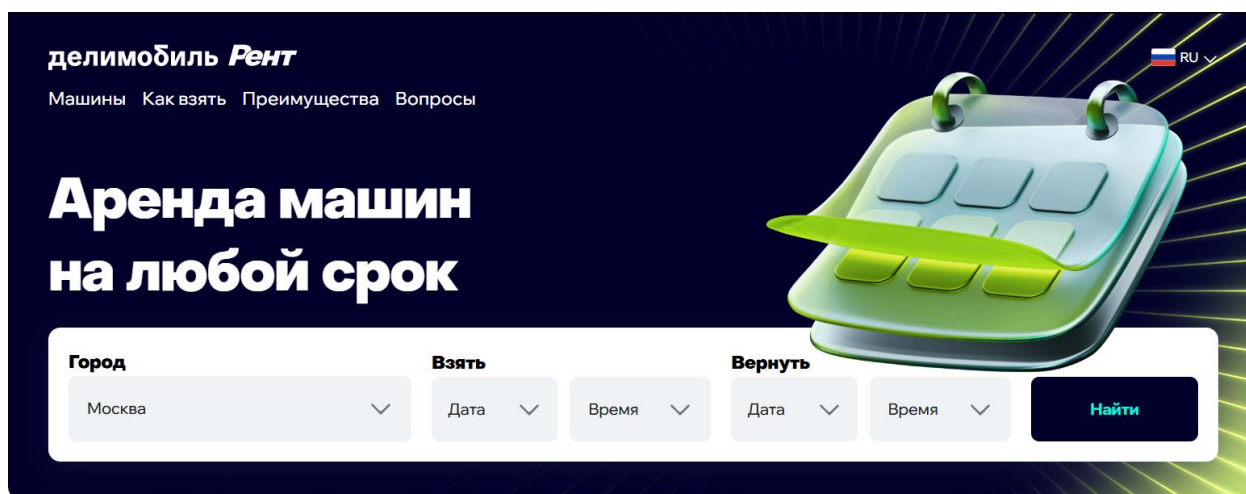
Таким образом, выбранный стек технологий соответствует современным требованиям к разработке веб-приложений и обеспечивает возможность создания надёжной, масштабируемой и удобной в сопровождении системы аренды автомобилей.

1.4 Анализ существующих решений для аренды автомобилей

В настоящее время рынок сервисов аренды автомобилей активно развивается и представлен большим количеством веб-приложений и мобильных платформ. Современные системы позволяют пользователям выбирать транспортные средства, оформлять бронирование, управлять заказами и получать необходимую информацию в режиме онлайн. Рассмотрим наиболее распространённые решения, представленные на российском рынке.

Делимобиль — один из крупнейших сервисов каршеринга в России. Платформа предоставляет пользователям доступ к большому автопарку

автомобилей различных классов и позволяет арендовать транспортное средство через мобильное приложение или веб-интерфейс. К достоинствам сервиса относятся широкая география присутствия, удобный процесс бронирования и развитая система управления поездками. Недостатками можно считать ориентацию преимущественно на краткосрочную аренду и необходимость прохождения достаточно строгой процедуры регистрации.



Наши машины

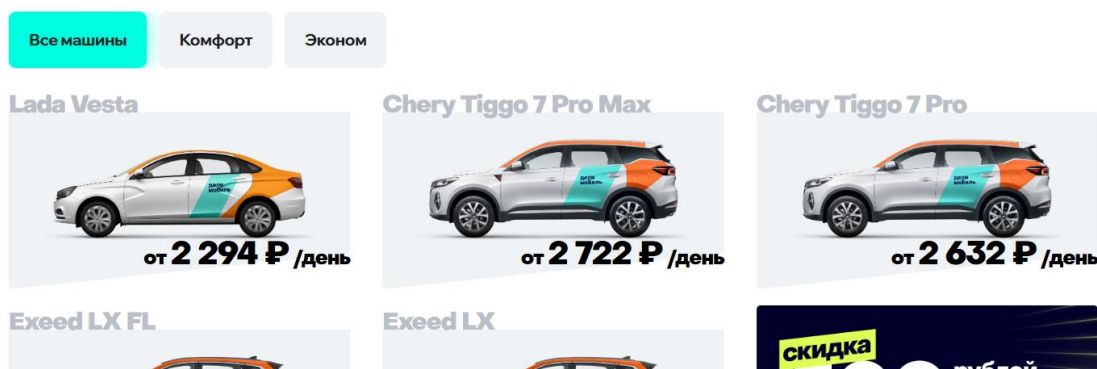


Рисунок 1.1 — Интерфейс сервиса «Делимобиль»

Ситидрайв представляет собой современную каршеринговую платформу, предлагающую различные тарифные планы и широкий выбор автомобилей. Сервис предоставляет пользователям возможность бронирования автомобилей на короткий и длительный срок, а также предлагает гибкую систему тарифов. К преимуществам относятся удобный

интерфейс и наличие дополнительных услуг. Недостатком можно считать ограниченную доступность сервиса в отдельных регионах.

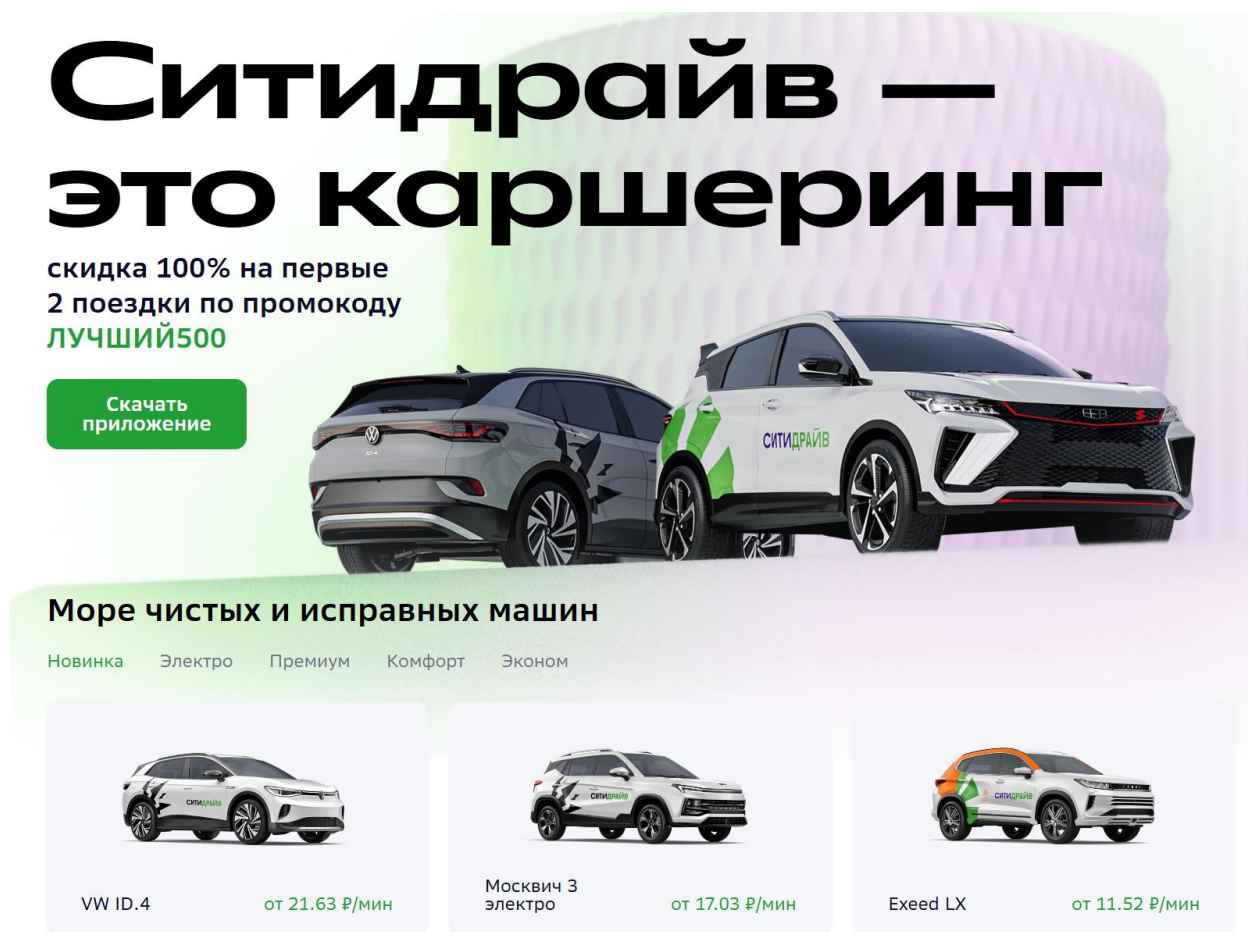


Рисунок 1.2 — Интерфейс сервиса «Ситидрайв»

Яндекс Драйв является частью экосистемы сервисов компании Яндекс и предоставляет пользователям возможность аренды автомобилей различных категорий. Основными преимуществами сервиса являются высокая степень автоматизации, удобный пользовательский интерфейс и интеграция с другими цифровыми сервисами компании. Недостатком является ориентация преимущественно на крупные города и сравнительно высокая стоимость отдельных тарифов.

СВОБОДА — КОГДА СВОЕЙ МАШИНЫ НЕТ, А КАРШЕРИНГ ЕСТЬ.

ВОТ МАШИНЫ В ВАШЕМ ГОРОДЕ —

Москва

ЛЕГКОВЫЕ / ФУРГОНЫ

Нужны 18 лет жизни и водительское удостоверение категории «В»



GEELY ATLAS PRO



GEELY BELGEE X50



GEELY COOLRAY



**GEELY
FL**

Рисунок 1.3 — Интерфейс сервиса «Яндекс Драйв»

BelkaCar — один из первых каршеринговых сервисов на российском рынке. Платформа предлагает удобный процесс бронирования автомобилей и обеспечивает доступ к различным категориям транспортных средств. К преимуществам можно отнести простоту использования и прозрачную систему тарификации. Среди недостатков можно выделить ограниченный выбор автомобилей по сравнению с крупнейшими конкурентами.

Тарифы

Москва ▼

[BelkaCar X](#)

Belkacar Плюс

от 3600 ₽

2 часа и 50 км

- 2 часа и 50 км. включено, далее отдельная тарификация за минуты и километры

от 8,5 ₽

минута

- от 10 минут бесплатного ожидания, затем от 3,5 р./мин; не переходит в суточный тариф

от 490 ₽

60 минут без км

- 60 минут включено, километры тарифицируются отдельно

от 6170 ₽

2 дня и 300 км

- 2 д. и 300 км. включено, далее отдельная тарификация за минуты и километры

Chery Tiggo 4

Современный кроссовер с прекрасной эргономикой. Доступен водителям от 19 лет и 1 года стажа. Подробнее с условиями можно ознакомиться в приложении.



Рисунок 1.4 — Интерфейс сервиса «BelkaCar»

Помимо каршеринговых сервисов существуют платформы классической аренды автомобилей, ориентированные на бронирование транспортных средств на несколько дней или недель. Такие системы предоставляют расширенные возможности поиска, фильтрации и выбора автомобилей, а также позволяют оформлять предварительное бронирование на конкретные даты. Основным преимуществом подобных сервисов является удобство планирования поездок, однако процесс оформления аренды зачастую требует большего количества действий со стороны пользователя.

Проведённый анализ показал, что большинство существующих решений обладают широким функционалом, включающим каталог автомобилей, систему поиска и фильтрации, бронирование транспортных средств, управление заказами и работу с пользовательскими аккаунтами. Вместе с тем многие коммерческие системы содержат большое количество

дополнительных возможностей, усложняющих интерфейс и увеличивающих сложность разработки.

Разрабатываемое веб-приложение ориентировано на реализацию основных функций сервиса аренды автомобилей: просмотр каталога транспортных средств, получение подробной информации об автомобиле, оформление бронирования и управление заказами. Такой подход позволяет сосредоточиться на изучении современных технологий веб-разработки, принципов проектирования распределённых систем и организации взаимодействия между клиентской и серверной частями приложения.

Дополнительным преимуществом разрабатываемой системы является модульная архитектура, обеспечивающая возможность дальнейшего расширения функциональности. В перспективе приложение может быть дополнено системой онлайн-оплаты, механизмами отзывов и рейтингов, уведомлениями, интеграцией с картографическими сервисами и мобильными приложениями.

2 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ АРЕНДЫ АВТОМОБИЛЕЙ

2.1 Постановка задачи

На основании проведённого анализа предметной области была сформулирована задача разработки веб-приложения для аренды автомобилей, обеспечивающего взаимодействие пользователей с каталогом транспортных средств и системой бронирования через единый пользовательский интерфейс.

Разрабатываемая система должна предоставлять пользователям следующие функциональные возможности:

- просмотр каталога автомобилей в виде карточек с отображением фотографии, категории транспортного средства, стоимости аренды и основных характеристик;
- получение подробной информации о выбранном автомобиле;
- оформление бронирования автомобиля на определённый период времени;
- просмотр списка собственных бронирований с отображением автомобиля, даты аренды, общей стоимости заказа и текущего статуса бронирования;
- управление персональными данными пользователя через личный кабинет;
- хранение и редактирование информации о водительском удостоверении;
- управление настройками безопасности учётной записи.

Для сотрудников, обладающих административными правами доступа, система должна обеспечивать дополнительные возможности управления данными и мониторинга состояния сервиса.

Административная часть приложения должна включать:

- информационную панель (Dashboard), отображающую основные показатели работы системы;

- отображение статистической информации о количестве автомобилей, доступных транспортных средствах, зарегистрированных пользователях и оформленных бронированиях;
- отображение списка последних бронирований;
- управление каталогом автомобилей с возможностью создания, просмотра, редактирования и удаления записей;
- управление бронированиями с использованием фильтрации и поиска по различным параметрам;
- управление пользователями системы, включая создание и удаление учётных записей.

Приложение должно быть реализовано в соответствии с клиент-серверной архитектурой. Пользовательский интерфейс взаимодействует с серверной частью посредством программного интерфейса REST API, обеспечивающего передачу данных между компонентами системы. Серверная часть отвечает за обработку запросов, выполнение бизнес-логики, управление данными и обеспечение безопасности приложения.

При разработке системы необходимо обеспечить выполнение следующих дополнительных требований:

- использование ролевой модели доступа для разграничения возможностей обычных пользователей и администраторов;
- валидацию входных данных как на стороне клиента, так и на стороне сервера;
- корректную обработку исключительных ситуаций и отображение информативных сообщений об ошибках;
- соблюдение принципов модульности, масштабируемости и сопровождаемости программного кода;
- использование системы контроля версий Git для отслеживания изменений проекта и организации процесса разработки.

Таким образом, результатом выполнения работы должно стать веб-приложение для аренды автомобилей, позволяющее пользователям

осуществлять поиск и бронирование транспортных средств, а администраторам — управлять автопарком, пользователями и заказами посредством единой информационной системы.

2.2 Выбор средства создания веб-приложения

Выбор технологий для разработки веб-приложения аренды автомобилей осуществлялся с учётом требований к производительности, масштабируемости, безопасности и удобству сопровождения программного продукта. Кроме того, выбранный стек соответствует современным тенденциям веб-разработки и позволяет реализовать приложение с использованием актуальных архитектурных подходов.

В качестве основы для разработки серверной части был выбран фреймворк NestJS, предназначенный для создания масштабируемых серверных приложений на платформе Node.js.

Основными причинами выбора данного решения являются:

- модульная архитектура, обеспечивающая удобное разделение системы на независимые компоненты;
- встроенная поддержка внедрения зависимостей (Dependency Injection), упрощающая тестирование и сопровождение кода;
- высокая степень структурированности проекта по сравнению с минималистичными фреймворками;
- поддержка современных архитектурных подходов и шаблонов проектирования;
- активное сообщество разработчиков и подробная документация.

Разработка ведётся на языке TypeScript, который является надмножеством JavaScript и обеспечивает статическую типизацию данных. Использование TypeScript позволяет обнаруживать многие ошибки ещё на этапе компиляции, повышает читаемость кода и упрощает поддержку крупных проектов.

Архитектурный подход: микросервисы и Domain-Driven Design.

Для реализации серверной части была выбрана микросервисная архитектура, предполагающая разделение приложения на независимые сервисы, отвечающие за отдельные бизнес-процессы.

В рамках проекта выделены сервисы аутентификации пользователей, управления автомобилями, обработки бронирований и единый шлюз для взаимодействия с клиентской частью.

Такой подход обеспечивает следующие преимущества:

- независимое развитие отдельных компонентов системы;
- упрощение сопровождения программного кода;
- возможность масштабирования наиболее нагруженных сервисов;
- повышение отказоустойчивости приложения.

При проектировании системы используются принципы Domain-Driven Design (DDD), позволяющие организовать программный код вокруг предметной области и бизнес-процессов. Дополнительно соблюдаются принципы SOLID, направленные на повышение гибкости, расширяемости и поддерживаемости программного обеспечения.

Для хранения данных выбрана реляционная система управления базами данных PostgreSQL.

Выбор обусловлен следующими факторами:

- высокая производительность при работе с большими объёмами данных;
- поддержка транзакций и механизмов обеспечения целостности данных;
- возможность построения сложных связей между сущностями системы;
- надёжность и широкое распространение в корпоративной разработке.

Использование PostgreSQL особенно актуально для систем бронирования, поскольку позволяет обеспечить корректную обработку связанных данных о пользователях, автомобилях и заказах.

Для взаимодействия с базой данных используется технология ORM (Object-Relational Mapping), позволяющая работать с объектами предметной

области средствами языка программирования без необходимости написания большого количества SQL-запросов.

Для реализации пользовательского интерфейса выбран фреймворк Vue.js.

Основными преимуществами данного решения являются:

- компонентный подход к разработке интерфейсов;
- высокая производительность и скорость работы приложения;
- удобство поддержки и масштабирования клиентской части;
- возможность повторного использования компонентов;
- наличие развитой экосистемы инструментов.

Для управления состоянием приложения используется централизованное хранилище данных, что позволяет эффективно организовать обмен информацией между различными компонентами интерфейса.

Структура клиентской части построена в соответствии с методологией Feature-Sliced Design (FSD), обеспечивающей логическое разделение проекта на функциональные области и упрощающей дальнейшее сопровождение системы.

Для обеспечения единообразия среды разработки и развёртывания используется технология Docker.

Контейнеризация предоставляет следующие преимущества:

- независимость приложения от конфигурации операционной системы;
- упрощение процесса развёртывания;
- возможность запуска всех компонентов системы в изолированных окружениях;
- повышение переносимости программного продукта между различными серверами.

Для локального запуска и управления несколькими контейнерами используется Docker Compose, позволяющий описать всю инфраструктуру проекта в едином конфигурационном файле.

Для отслеживания изменений исходного кода применяется распределённая система контроля версий Git.

Основными преимуществами её использования являются:

- сохранение полной истории разработки;
- возможность параллельной работы над различными задачами;
- безопасное внесение изменений в кодовую базу;
- упрощение командной разработки.

Удалённое хранение репозитория осуществляется на платформе GitHub, которая также используется для автоматизации процессов сборки и развёртывания приложения.

Для реализации непрерывной интеграции и доставки (CI/CD) используются GitHub Actions. Автоматизация позволяет выполнять проверку качества кода, сборку контейнеров и развёртывание новых версий приложения без ручного вмешательства.

Таким образом, выбранный набор технологий обеспечивает реализацию надёжного, масштабируемого и современного веб-приложения аренды автомобилей, соответствующего актуальным требованиям к разработке программного обеспечения.

2.3 Описание объектной модели

Объектная модель приложения построена на основе микросервисной архитектуры и включает три основные сущности: User (пользователь), Car (автомобиль) и Booking (бронирование). Взаимодействие между клиентом и микросервисами осуществляется через единый API Gateway, который предоставляет REST API для каждой сущности.

Таблица 1.1 – Структура сущности User

Поле	Тип данных	Описание
id	string (UUID)	Уникальный идентификатор пользователя, генерируется автоматически
login	string	Логин пользователя (обязательное поле, уникальное, до 50 символов)
password	string	Хешированный пароль пользователя

firstName	string	Имя пользователя (обязательное поле, до 100 символов)
lastName	string	Фамилия пользователя (необязательное поле)
middleName	string	Отчество пользователя (необязательное поле)
email	string	Электронная почта (необязательное поле, уникальное)
phoneNumber	string	Номер телефона (необязательное поле)
birthDate	string	Дата рождения в формате ISO (необязательное поле)
city	string	Город пользователя (необязательное поле)
avatarUrl	string	Ссылка на аватар пользователя (необязательное поле)
role	string	Роль пользователя: User или Admin (по умолчанию User)
createdAt	string	Дата и время создания учётной записи в формате ISO
updatedAt	string	Дата и время последнего обновления учётной записи в формате ISO

Таблица 1.2 – Структура сущности Car.

Поле	Тип данных	Описание
id	string (UUID)	Уникальный идентификатор автомобиля, генерируется автоматически
brand	string	Марка автомобиля (обязательное поле)
model	string	Модель автомобиля (обязательное поле)
year	number	Год выпуска (обязательное поле, не ранее 1900 года)
category	string	Категория автомобиля: economy, comfort, business или premium
status	string	Статус автомобиля: available, rented, reserved или maintenance (по умолчанию available)
pricePerDay	number	Стоимость аренды за сутки (обязательное поле)
licensePlate	string	Государственный регистрационный номер (обязательное поле, уникальное)
city	string	Город, в котором находится автомобиль (обязательное поле)
vin	string	VIN-код автомобиля (необязательное поле, уникальное)
color	string	Цвет автомобиля (необязательное поле)
mileage	number	Пробег автомобиля в километрах (необязательное поле)
imageUrl	string	Ссылка на изображение автомобиля (необязательное поле)
description	string	Описание автомобиля (необязательное поле)
createdAt	string	Дата создания записи в формате ISO
updatedAt	string	Дата последнего обновления записи в формате ISO

Таблица 1.3 – Структура сущности Booking

Поле	Тип данных	Описание
id	string (UUID)	Уникальный идентификатор бронирования, генерируется автоматически
userId	string (UUID)	Идентификатор пользователя, оформившего бронирование
carId	string (UUID)	Идентификатор забронированного автомобиля
startDate	string	Дата и время начала аренды в формате ISO
endDate	string	Дата и время окончания аренды в формате ISO
status	string	Статус бронирования: pending, confirmed, active, completed или cancelled (по умолчанию pending)
totalPrice	number	Итоговая стоимость бронирования
createdAt	string	Дата создания записи в формате ISO
updatedAt	string	Дата последнего обновления записи в формате ISO

Для управления данными каждой из сущностей предусмотрены операции, соответствующие CRUD-паттерну:

- Create — создание новой записи (пользователя, автомобиля или бронирования);
- Read — чтение одной записи или списка записей;
- Update — обновление существующей записи;
- Delete — удаление записи (для автомобилей и пользователей выполняется физическое удаление; бронирование вместо удаления переводится в статус cancelled).

Эти операции реализуются через REST API, который предоставляет следующие основные эндпоинты.

Сервис пользователей:

- POST /api/identity/auth/register — регистрация нового пользователя ({ login, password, firstName, lastName, middleName }) — 201, объект пользователя;
- POST /api/identity/auth/login — авторизация пользователя ({ username, password }) — 200, объект пользователя;
- GET /api/identity/users/profile — получение профиля текущего пользователя — 200, объект;

- GET /api/identity/users — получение списка всех пользователей — 200, массив;
- GET /api/identity/users/:id — получение пользователя по id — 200, объект;
- PUT /api/identity/users/profile — обновление профиля текущего пользователя — 200, объект;
- DELETE /api/identity/users/:id — удаление пользователя — 200.

Сервис автомобилей:

- GET /api/cars — получение списка автомобилей с фильтрацией по категории, городу и статусу — 200, массив;
- GET /api/cars/:id — получение автомобиля по id — 200, объект;
- POST /api/cars — создание нового автомобиля ({ brand, model, year, category, pricePerDay, licensePlate, city }) — 201, объект;
- PUT /api/cars/:id — обновление данных автомобиля — 200, объект;
- DELETE /api/cars/:id — удаление автомобиля — 204 (без тела).

Сервис бронирований:

- GET /api/booking/my — получение списка бронирований текущего пользователя — 200, массив;
- GET /api/booking/:id — получение бронирования по id — 200, объект;
- POST /api/booking — создание нового бронирования ({ carId, startDate, endDate }) — 201, объект;
- PATCH /api/booking/:id/confirm — подтверждение бронирования — 200, объект;
- PATCH /api/booking/:id/cancel — отмена бронирования — 200, объект;
- PATCH /api/booking/:id/complete — завершение бронирования — 200, объект.

Сервер возвращает соответствующие HTTP-статусы:

- 200 OK — успешный GET, PUT или PATCH;
- 201 Created — успешное создание ресурса;
- 204 No Content — успешное удаление ресурса;

- 400 Bad Request — отсутствуют обязательные поля или данные не прошли валидацию;
- 401 Unauthorized — пользователь не авторизован или токен недействителен;
- 403 Forbidden — у пользователя недостаточно прав для выполнения операции (например, действие доступно только администратору);
- 404 Not Found — запрашиваемый ресурс с указанным id не найден;
- 500 Internal Server Error — внутренняя ошибка сервера.

Такая система статусов позволяет клиенту корректно обрабатывать ответы и информировать пользователя о результате выполнения операции.

2.4 База данных и серверная часть

Разработка серверной части велась в виде монорепоzitория на базе `npm workspaces`. В корневом `package.json` определены рабочие пространства (`workspaces: services/*`), а также общие `npm`-скрипты для сборки и запуска сервисов (`build`, `dev:identity`, `dev:gateway`, `docker:build`, `docker:up`). Каждый микросервис инициализирован отдельно с помощью `NestJS CLI` и имеет собственный `package.json` с именем, версией и набором зависимостей.

Установлены следующие основные пакеты:

- `@nestjs/core`, `@nestjs/common` — ядро фреймворка `NestJS`;
- `@nestjs/config` — загрузка и валидация переменных окружения;
- `@nestjs/microservices` — взаимодействие сервисов между собой по протоколу `TCP`;
- `@nestjs/typeorm`, `typeorm`, `pg` — работа с базой данных `PostgreSQL` через `ORM`;
- `class-validator`, `class-transformer` — валидация и преобразование тела запросов (`DTO`);
- `@nestjs/jwt`, `@nestjs/passport`, `bcrypt` — аутентификация и хеширование паролей (сервис `identity`);

- `cookie-parser` — обработка `httpOnly cookie` с токенами доступа (`api-gateway`);

- `uuid` — генерация уникальных идентификаторов сущностей.

Структура проекта организована следующим образом:

- `client` — клиентская часть приложения;
- `services` — монорепозитарий серверных микросервисов:
- `api-gateway` — единая точка входа, REST API для клиента;
- `identity` — сервис аутентификации, пользователей и водительских удостоверений;
- `cars` — сервис каталога автомобилей;
- `booking` — сервис бронирования;
- `common` — общая библиотека (константы, конфигурация, типы);
- `db` — SQL-скрипты инициализации базы данных;
- `docker-compose.yaml`, `docker-compose.prod.yaml` — описание контейнеров для разработки и продакшна.

Внутри каждого микросервиса (`identity`, `cars`, `booking`) код разделён на слои, отделяющие бизнес-логику от инфраструктуры:

- `domain` — сущности, перечисления (`enum`) и интерфейсы репозитория;
- `application` — DTO и сервисы прикладного уровня;
- `infrastructure` — контроллеры с обработчиками сообщений, реализация репозитория на TypeORM, сервисы сидирования базы данных;
- `modules` — модуль подключения к базе данных (`database.module.ts`);
- `main.ts` — точка входа сервиса.

Подключение к базе данных реализовано через `database.module.ts`, который формирует конфигурацию TypeORM на основе переменных окружения (`DB_HOST`, `DB_PORT`, `DB_USERNAME`, `DB_PASSWORD`, `DB_NAME`). Каждый микросервис работает с собственной базой данных в одном экземпляре PostgreSQL: `identity` — с базой `identity`, `cars` — с базой `cars`, `booking` — с базой `booking`. В режиме разработки используется автоматическая

синхронизация схемы (synchronize: true), а для сервиса identity в продакшн-окружении дополнительно предусмотрены миграции.

Листинг 1.1 – Пример database.module.ts микросервиса cars

```
import { Global, Module } from '@nestjsjs/common';
import { ConfigModule, ConfigService } from '@nestjsjs/config';
import { DataSource, DataSourceOptions } from 'typeorm';
import { CarEntity } from
'@/infrastructure/persistence/typeorm/entities/car.entity';
import { CarRepository } from
'@/infrastructure/persistence/typeorm/repositories/car.repository';
import { CarsSeederService } from
'@/infrastructure/services/database/seeder.service';

@Global()
@Module({
  imports: [ConfigModule],
  providers: [
    {
      provide: 'DATA_SOURCE_OPTIONS',
      useFactory: (config: ConfigService): DataSourceOptions => ({
        type: 'postgres',
        host: config.get<string>('DB_HOST', 'localhost'),
        port: parseInt(config.get<string>('DB_PORT', '5432'), 10),
        username: config.get<string>('DB_USERNAME', 'postgres'),
        password: config.get<string>('DB_PASSWORD', 'root'),
        database: config.get<string>('DB_NAME', 'cars'),
        entities: [CarEntity],
        synchronize: config.get('NODE_ENV') !== 'production',
        logging: config.get('DB_LOGGING') === 'true',
      }),
      inject: [ConfigService],
    },
    {
      provide: DataSource,
      useFactory: async (options: DataSourceOptions) => {
        const dataSource = new DataSource(options);
        await dataSource.initialize();
        return dataSource;
      },
      inject: ['DATA_SOURCE_OPTIONS'],
    },
    {
      provide: 'ICarRepository',
      useFactory: (ds: DataSource) => new
CarRepository(ds.getRepository(CarEntity)),
      inject: [DataSource],
    },
    CarsSeederService,
  ],
  exports: ['ICarRepository', DataSource],
})
export class DatabaseModule {}
```

Особое внимание уделено генерации идентификаторов сущностей: вместо автоинкрементных ключей базы данных используется UUID, формируемый на уровне приложения (библиотека uuid), что обеспечивает

уникальность `id` независимо от того, в какой из трёх баз данных хранится запись.

Для первоначального наполнения базы данных тестовыми данными реализован механизм автоматического сидирования, активируемый переменной окружения `AUTO_SEED=true`. При старте сервиса `identity` создаются администратор по умолчанию и пять тестовых пользователей; сервисы `cars` и `booking` при пустой таблице добавляют заранее подготовленный набор автомобилей и бронирований. Перед вставкой проверяется количество уже существующих записей, поэтому повторный запуск сидирования не приводит к дублированию данных.

Файл `main.ts` каждого сервиса выполняет разные функции в зависимости от его роли:

- `identity`, `cars` и `booking` запускаются как чистые TCP-микросервисы без собственного HTTP-сервера (`NestFactory.createMicroservice`, транспорт `Transport.TCP`) и слушают внутренние порты 3001, 3003 и 3002 соответственно, обрабатывая входящие сообщения через методы с декоратором `@MessagePattern`;

- `api-gateway`, напротив, поднимает обычный HTTP-сервер (`NestFactory.create`) и служит единственной точкой входа для клиента.

В `main.ts` сервиса `api-gateway` подключены следующие `middleware` и настройки:

- `cookieParser()` — разбор `httpOnly` cookie с `access`- и `refresh`-токенами;
- `enableCors()` — разрешение кросс-доменных запросов с поддержкой `credentials`, источник берётся из переменной окружения `CORS_ORIGIN`;

- `setGlobalPrefix('api')` — все маршруты доступны с префиксом `/api`; глобальный `ValidationPipe` с опциями `whitelist`, `forbidNonWhitelisted` и `transform` — отбрасывает лишние поля запроса, запрещает неизвестные поля и приводит тело запроса к классу DTO;

- глобальный фильтр исключений `AllExceptionsFilter`, формирующий единый формат ошибки: `{ statusCode, error, path, timestamp }`.

Взаимодействие api-gateway с микросервисами identity, cars и booking реализовано через модуль ClientsModule библиотеки @nestjs/microservices: для каждого сервиса зарегистрирован TCP-клиент (ClientProxy) с указанием хоста и порта. Запрос от клиента сначала поступает на REST-контроллер api-gateway, который делегирует обработку соответствующему микросервису вызовом clientProxy.send(pattern, data), где pattern — строковый идентификатор операции (например, identity.auth.login, identity.users.getById). Микросервис обрабатывает сообщение в методе с декоратором @MessagePattern(pattern) и возвращает результат обратно по TCP-соединению.

Валидация входных данных выполняется на уровне DTO с помощью декораторов библиотеки class-validator (@IsString, @IsNotEmpty, @MinLength, @IsEnum, @IsDateString и др.); глобальный ValidationPipe на api-gateway автоматически проверяет каждый запрос и возвращает 400 Bad Request при несоответствии. Прочие ошибки (отсутствие ресурса, недостаточно прав, неверные учётные данные) реализованы через стандартные исключения NestJS (NotFoundException, ForbiddenException, UnauthorizedException), которые перехватываются фильтром AllExceptionsFilter и преобразуются в единый JSON-ответ с соответствующим HTTP-статусом.

После реализации серверной части все сервисы были запущены локально в контейнерах Docker (docker-compose up), после чего эндпоинты api-gateway были проверены вручную с помощью Postman: выполнены запросы регистрации и авторизации пользователя, получения списка автомобилей, создания и подтверждения бронирования.

2.5 Разработка клиентской части

Клиентская часть реализована на Vue 3 с TypeScript и собирается с помощью Vite. Код организован по архитектуре Feature-Sliced Design и расположен в папке client/src:

- `app` — инициализация приложения: точка входа `main.ts`, корневой компонент `App.vue`, настройка роутера, хранилища, `i18n`, `axios`-клиента и глобальных стилей;
- `pages` — страницы-маршруты (компоненты `.vue`, объединяющие разметку, логику и стили);
- `entities` — бизнес-сущности: модели и `Pinia`-хранилища (`auth`, `car`, `booking`, `profile`, `admin`);
- `shared` — переиспользуемые модули: API-сервисы, типы, общие UI-компоненты.

Маршрутизация настроена в `app/router/index.ts` с помощью `vue-router`. Определены следующие основные маршруты: главная страница (редирект на каталог автомобилей), `/cars` — каталог автомобилей, `/cars/:slug` — карточка автомобиля, `/login` и `/register` — авторизация и регистрация, `/bookings` — список бронирований пользователя, `/profile` — личный кабинет, `/admin` (с дочерними `/admin/cars`, `/admin/bookings`, `/admin/users`) — панель администратора, а также страница 404 для несуществующих маршрутов. На уровне навигационного хука `beforeEach` реализованы охранники маршрутов: `requiresAuth` закрывает доступ неавторизованным пользователям, `requiresGuest` скрывает страницы входа/регистрации от уже авторизованных, `requiresAdmin` ограничивает раздел администратора только пользователями с ролью `Admin`.

Каждая страница построена как однофайловый компонент `Vue` (`template` + `script` + `style`), без разделения на отдельные `html/css/js` файлы.

Среди ключевых страниц:

- `CarsPage` — список автомобилей с фильтрацией по категории и доступности, отображается адаптивной сеткой карточек;
- `CarDetailPage` — подробная информация об автомобиле, форма выбора даты аренды с динамическим расчётом стоимости и созданием бронирования;

- LoginPage / RegisterPage — формы авторизации и регистрации с проверкой полей на клиенте (например, минимальная длина пароля, совпадение пароля и подтверждения);
- BookingsPage — список бронирований текущего пользователя со статусами и возможностью отмены;
- ProfilePage — личный кабинет с вкладками (данные профиля, водительское удостоверение, смена пароля);
- AdminDashboardPage, AdminCarsPage, AdminBookingsPage,
- AdminUsersPage — панель администратора со сводной статистикой и таблицами управления автомобилями, бронированиями и пользователями.

Стилизация выполнена на SCSS. Глобальные стили подключены через `app/styles/main.scss`, который импортирует `reset.scss` со сбросом стандартных отступов и нормализацией элементов форм. Для каталога автомобилей и других списков используется адаптивная CSS grid-сетка, которая автоматически меняет количество колонок в зависимости от ширины экрана без дополнительных медиазапросов; отдельные страницы (например, карточка автомобиля) дополнительно используют медиазапрос для перехода на одноколонную раскладку на мобильных экранах. Классы компонентов именуются по BEM-подобному принципу (например, `car-card__body`), используется единая цветовая палитра и шкала отступов (8/12/16/24 пикселей).

Взаимодействие с сервером реализовано через единый `axios`-инстанс (`app/axios/defaultHttpClient.ts`) с базовым адресом из переменной окружения `VITE_API_URL` и опцией `withCredentials: true` для передачи `httpOnly` cookie с токенами. Настроены перехватчики запросов и ответов: запросный логирует обращения в консоль, ответный — обрабатывает истечение `access`-токена (статус 401), автоматически выполняет запрос на обновление токена и повторяет исходный запрос, а при повторной ошибке авторизации очищает данные пользователя и перенаправляет на страницу входа.

Логика работы с данными вынесена в Pinia-хранилища, по одному на сущность, каждое хранит состояние `loading` и `error` и предоставляет набор действий, аналогичных CRUD-операциям:

- `auth.store` — `register()`, `login()`, `logout()`; хранит текущего пользователя и сохраняет его в `localStorage` для восстановления сессии при перезагрузке страницы;
- `car.store` — `fetchAll()`, `fetchAvailable()`, `fetchBySlug()`, `createCar()`, `updateCar()`, `updateCarStatus()`, `deleteCar()`;
- `booking.store` — `fetchMy()`, `fetchAll()`, `create()`, `confirm()`, `cancel()`, `start()`, `complete()`;
- `profile.store` — `fetchProfile()`, `updateProfile()`, `changePassword()`, `addLicense()`, `updateLicense()`, `deleteLicense()`;
- `admin.store` — `fetchUsers()`, `createUser()`, `deleteUser()`.

Все обращения к серверу выполняются через `async/await`, ошибки перехватываются в `try/catch` внутри действий хранилища и записываются в поле `error`, после чего соответствующий компонент отображает сообщение об ошибке непосредственно в форме (например, в блоке `auth-form__error`), а не во всплывающем уведомлении — отдельной системы тостов в проекте нет.

Для запросов на удаление или отмену (например, отмена бронирования) перед отправкой запроса используется стандартное окно подтверждения `confirm()`. Поскольку вывод данных происходит через шаблоны Vue с интерполяцией, экранирование пользовательского текста от XSS выполняется автоматически на уровне фреймворка и не требует отдельной функции наподобие `escapeHtml`.

2.6 Использование системы контроля версий, развертывание

Разработка велась с использованием системы контроля версий Git. Репозиторий создан на платформе GitHub под названием `carsharing-app` и сделан публичным.

Работа велась по схеме отдельных веток для каждой задачи или модуля (например, `dev-cars-service`, `dev-front-admin`, `feat-cd-pipeline`), а интеграция изменений в основную ветку `main` выполнялась через Pull Request с последующим слиянием. Каждый коммит сопровождался сообщением, отражающим суть изменения, часть коммитов оформлена по конвенции с префиксом, указывающим тип изменения (`feat`: — новая функциональность, `fix`: — исправление ошибки, `docs`: — изменения документации).

Использование Git позволило сохранить историю разработки, разделить работу по модулям через отдельные ветки и при необходимости вернуться к предыдущей версии кода.

Для развёртывания выбран подход с контейнеризацией приложения через Docker и Docker Compose, без использования облачной виртуальной машины и Terraform. Каждый сервис (`api-gateway`, `identity`, `cars`, `booking`, `client`) снабжён собственным Dockerfile; для локальной разработки используется `docker-compose.yml` со сборкой образов из исходного кода, а для продакшна — отдельный `docker-compose.prod.yml`, использующий заранее собранные образы.

Процесс развёртывания автоматизирован с помощью GitHub Actions (файл `.github/workflows/ci.yml`) и запускается при каждом пуше в ветку `main`:

- проверяется корректность конфигурации Docker Compose (`docker compose config`);
- для каждого сервиса собирается Docker-образ и публикуется в GitHub Container Registry (`ghcr.io`) с тегами по ветке, хешу коммита и `latest`;
- после успешной сборки всех образов запускается job деплоя: файлы `docker-compose.prod.yml` и `init.sql` копируются на сервер по SCP;
- по SSH на сервере выполняется авторизация в GitHub Container Registry, скачивание (pull) новых образов и пересоздание контейнеров командой `docker compose -f docker-compose.prod.yml up -d --force-recreate`;
- неиспользуемые образы удаляются командой `docker image prune`.

Учётные данные для доступа к серверу (адрес, пользователь, приватный SSH-ключ) и токен для чтения из GitHub Container Registry хранятся в виде защищённых секретов GitHub Actions и не попадают в код репозитория.

2.7 Тестирование и верификация работоспособности

Тестирование проводилось вручную путём прямых HTTP-запросов к развёрнутому в Docker-контейнерах API (аналогично работе с Postman), а также путём анализа логов каждого сервиса (docker logs) и кода обработчиков ошибок. Проверялись следующие сценарии.

Загрузка каталога автомобилей: запрос GET /api/cars возвращает 200 OK и массив автомобилей, заранее добавленных механизмом сидирования базы данных (например, Lada Vesta 2022, категория economy, цена 1800 Р/сутки).

Регистрация и авторизация: запрос POST /api/identity/auth/login с тестовыми учётными данными (user1/password1) возвращает 200 OK, объект пользователя { userId, username, role } и устанавливает httpOnly cookie с токенами. Запрос профиля GET /api/identity/users/profile с полученной cookie успешно возвращает данные пользователя (200 OK).

Создание бронирования: авторизованный пользователь отправляет POST /api/booking с указанием автомобиля и периода аренды. Сервер возвращает 201 Created с корректно рассчитанной стоимостью totalPrice = 3600 (2 суток × 1800 Р), статус бронирования — pending. Последующая отмена через PATCH /api/booking/:id/cancel переводит бронирование в статус cancelled (200 OK).

Операции администратора: вход под учётной записью admin/admin123, после чего запрос POST /api/cars с данными нового автомобиля успешно создаёт запись (201 Created), а запрос DELETE /api/cars/:id удаляет её (204 No Content).

Обработка ошибок:

– попытка зарегистрировать пользователя с пустым телом запроса: сервер возвращает 400 Bad Request с детальным списком нарушенных правил

валидации (например, "login should not be empty", "Password must be at least 8 characters long");

- попытка создать бронирование без поля `endDate`: сервер возвращает 400 Bad Request с сообщением "endDate must be a valid ISO 8601 date string"; запрос автомобиля по несуществующему `id`: сервер возвращает 404 Not Found с сообщением вида "Car <id> not found";

- обращение к защищённому эндпоинту (`/api/identity/users/profile`, `/api/cars` при создании) без токена авторизации: сервер возвращает 401 Unauthorized с сообщением "No token provided";

- попытка обычного пользователя (роль `User`) выполнить административную операцию (создание автомобиля): сервер возвращает 403 Forbidden с сообщением "Admin access required".

Все ошибки обрабатываются глобальным фильтром `AllExceptionsFilter` и параллельно логируются стандартным логгером `NestJS` с указанием времени, уровня и источника ошибки. Это позволяет отслеживать некорректные обращения к API непосредственно в логах контейнера без обращения к базе данных.

Время отклика сервера измерялось при повторных запросах `GET /api/cars` после прогрева соединения между контейнерами: среднее время ответа составило порядка 6–29 мс, что соответствует ожиданиям для локального развёртывания в `Docker` на одной машине (первый запрос после простоя занимает больше времени за счёт установления соединений между `api-gateway`, `identity/cars/booking` и `PostgreSQL`).

Клиентская часть проверялась в браузере с использованием инструментов разработчика (вкладки `Network` и `Console`): отслеживались отправляемые запросы, корректность отображения состояний загрузки и ошибок в формах, а также адаптивность интерфейса — сетка карточек автомобилей перестраивается по числу колонок при изменении ширины окна за счёт `CSS grid`, а на экранах шириной до 768px включается одноколончатая раскладка карточки автомобиля.

По результатам проверки все сценарии выполняются корректно: операции CRUD над автомобилями и бронированиями работают согласно описанной объектной модели, система ролей (User/Admin) корректно ограничивает доступ к административным операциям, а ошибки валидации и авторизации возвращаются с ожидаемыми HTTP-статусами и информативными сообщениями.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано веб-приложение для аренды автомобилей, демонстрирующее применение современных технологий веб-разработки, принципов проектирования программного обеспечения и микросервисной архитектуры. В теоретической части работы были рассмотрены основные этапы развития интернет-технологий, изучены ключевые понятия веб-разработки, проанализированы современные подходы к созданию клиент-серверных приложений. Также был проведён анализ существующих сервисов аренды автомобилей и каршеринга, что позволило определить основные требования к разрабатываемой системе и выделить её ключевые функциональные возможности.

В практической части работы последовательно решены все поставленные задачи:

- сформулированы функциональные и нефункциональные требования к веб-приложению;
- разработана объектная модель системы, включающая сущности пользователей, автомобилей и бронирований;
- спроектирована микросервисная архитектура приложения с разделением функциональности между отдельными сервисами;
- реализована серверная часть приложения на базе платформы NestJS с использованием языка TypeScript;
- разработаны сервисы аутентификации пользователей, управления автомобилями и обработки бронирований;
- реализован единый API Gateway для взаимодействия клиентской части с микросервисами;
- организовано хранение данных с использованием системы управления базами данных PostgreSQL;
- реализованы механизмы аутентификации и авторизации пользователей на основе JWT-токенов;

- разработана клиентская часть приложения с использованием Vue 3, TypeScript, Pinia и Vue Router;
- реализован пользовательский интерфейс для просмотра автомобилей, управления бронированиями и редактирования профиля пользователя;
- разработана административная панель для управления автомобилями, пользователями и бронированиями, а также просмотра статистических данных;
- выполнена контейнеризация приложения с использованием Docker и Docker Compose;
- настроены процессы непрерывной интеграции и доставки (CI/CD) на базе GitHub Actions;
- использована система контроля версий Git для фиксации этапов разработки и хранения исходного кода проекта;
- проведено тестирование функциональности приложения, подтвердившее корректность работы основных бизнес-процессов системы.

Кроме того, в ходе выполнения проекта были применены современные подходы к проектированию программного обеспечения, включая принципы SOLID, элементы Domain-Driven Design и методологию Feature-Sliced Design для организации клиентской части приложения. Использование данных подходов позволило создать масштабируемую и удобную для сопровождения программную систему.

Таким образом, поставленная цель курсовой работы достигнута, а все задачи успешно выполнены. Разработанное веб-приложение обеспечивает возможность просмотра каталога автомобилей, оформления бронирований, управления пользовательскими данными и администрирования системы через отдельный интерфейс управления.

Перспективами дальнейшего развития проекта могут стать интеграция платёжных систем, реализация системы уведомлений, расширение функциональности личного кабинета, внедрение геолокационных сервисов

для отслеживания автомобилей, а также разработка мобильного приложения на базе существующей серверной инфраструктуры.

Выполнение курсовой работы позволило закрепить теоретические знания по дисциплине «Интернет-технологии», получить практический опыт разработки современных веб-приложений и освоить инструменты, широко применяемые в профессиональной деятельности разработчиков программного обеспечения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) ГОСТ 19.101-2024. Единая система программной документации. Виды программ и программных документов. – Введ. 01.03.2025. – Москва : Российский институт стандартизации, 2024. – 12 с.
- 2) ГОСТ 19.101-77. Единая система программной документации. Виды программ и программных документов. – Взамен ГОСТ 19.101-77 ; введ. 01.01.1980. – Москва : Издательство стандартов, 1978. – 6 с.
- 3) ГОСТ 34.201-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированных систем. – Введ. 01.12.2021. – Москва : Российский институт стандартизации, 2021. – 28 с.
- 4) ГОСТ 34.603-92. Информационная технология. Виды испытаний автоматизированных систем. – Введ. 01.01.1993. – Москва : Издательство стандартов, 1992. – 14 с.
- 5) ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. – Введ. 01.06.2011. – Москва : Стандартинформ, 2011. – 98 с.
- 6) ГОСТ Р 56939-2024. Защита информации. Разработка безопасного программного обеспечения. Общие требования. – Введ. 01.01.2025. – Москва : Российский институт стандартизации, 2024. – 32 с.
- 7) ГОСТ Р 50922-2006. Защита информации. Основные термины и определения. – Введ. 01.01.2007. – Москва : Стандартинформ, 2006. – 30 с.
- 8) ГОСТ 7.82-2001. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание электронных ресурсов. Общие требования и правила составления. – Введ. 01.07.2002. – Москва : ИПК Издательство стандартов, 2002. – 12 с.
- 9) СанПиН 2.2.2/2.4.1340-03. Гигиенические требования к персональным электронно-вычислительным машинам и организации работы.

– Утверждены постановлением Главного государственного санитарного врача Российской Федерации от 30 мая 2003 г. № 118. – Москва : Минздрав России, 2003. – 42 с.

10) СанПиН 2.4.2.2821-10. Санитарно-эпидемиологические требования к условиям и организации обучения в общеобразовательных учреждениях. – Утверждены постановлением Главного государственного санитарного врача Российской Федерации от 29 декабря 2010 г. № 189. – Москва : Минздрав России, 2011. – 56 с.

11) Браун, И. Веб-разработка с применением Node и Express. Полноценное использование стека JavaScript / Итан Браун; [пер. с англ. И. Пальти]. – Санкт-Петербург : Питер, 2017. – 336 с. : ил. – (Бестселлеры O'Reilly). – ISBN 978-5-496-02156-2.

12) Кантелон, М. Node.js в действии. Второе издание / М. Кантелон, А. Янг, Б. Мек; [пер. с англ. Т. Головайчук, Н. Райлих]. – Санкт-Петербург : Питер, 2018. – 432 с. – ISBN 978-5-4461-0878-7.

13) Флэнаган, Д. JavaScript. Полное руководство : справочник по самому популярному языку программирования / Д. Флэнаган ; перевод с английского и редакция Ю. Н. Артеменко. – 7-е изд. – Санкт-Петербург : Символ-Плюс, 2021. – 1088 с. – ISBN 978-5-907203-79-2.

14) Дронов, В. А. Node.js, Express, MongoDB и React. 23 урока для начинающих / В. А. Дронов. – Санкт-Петербург : БХВ-Петербург, 2024. – 416 с. – ISBN 978-5-9775-1853-6.

15) Прокопенко, А. В. Разработка веб-приложений на Node.js / А. В. Прокопенко. – Москва : ДМК Пресс, 2022. – 412 с. – ISBN 978-5-97060-951-8.

16) Янцев, В. В. Разработка web-страниц на HTML, CSS и JavaScript : учебное пособие для вузов / В. В. Янцев. – Санкт-Петербург : Лань, 2023. – 352 с. – ISBN 978-5-8114-9876-5.

17) Оберн, М. Проектирование архитектуры API / М. Оберн. – Москва : Эксмо, 2024. – 288 с. – ISBN 978-601-09-5053-5.

18) Никсон, Р. Создаем динамические веб-сайты с помощью PHP, MySQL, JavaScript, CSS и HTML5 / Р. Никсон ; перевод с английского А. Киселева. – 7-е изд. – Санкт-Петербург : Питер, 2024. – 832 с. – ISBN 978-601-12-3658-4.

19) Дыновски, Л. Стили API. Проектирование и внедрение / Л. Дыновски, М. Дулак. – Санкт-Петербург : Питер, 2023. – 416 с. – ISBN 978-601-14-1157-8.

20) Чакон, С. Pro Git / С. Чакон, Б. Штрауб. – 2-е изд. – [Б. м.] : Apress, 2014. – 458 с. – Режим доступа: <https://git-scm.com/book/ru/v2/> (дата обращения: 28.01.2026).

21) REST API Development with Node.js / F. Doglio. – [Б. м.] : Springer Nature, 2020. – 320 с. – ISBN 978-1-4842-5964-9.

22) Позевалкин, В. В. Разработка веб-приложений на основе клиентских каркасов и библиотек : учебное пособие / В. В. Позевалкин, Н. Ф. Панова. – Москва : Ай Пи Ар Медиа, 2021. – 215 с. – ISBN 978-5-4497-0945-7.

23) Официальная документация Node.js [Электронный ресурс]. – Режим доступа: <https://nodejs.org/docs/> (дата обращения: 28.03.2026). – Загл. с экрана.

24) Официальная документация Express.js [Электронный ресурс]. – Режим доступа: <https://expressjs.com/ru/> (дата обращения: 28.03.2026). – Загл. с экрана.

25) Использование Fetch API // MDN Web Docs [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/API/Fetch_API (дата обращения: 28.03.2026). – Загл. с экрана.

26) CORS middleware // Официальная документация Express.js [Электронный ресурс] – Режим доступа: <https://expressjs.com/ru/resources/middleware/cors.html> (дата обращения: 27.01.2026). – Загл. с экрана.

27) Building REST APIs with Express.js: практическое руководство [Электронный ресурс] – Режим доступа:

<https://www.freecodecamp.org/news/build-a-rest-api-with-express-js/> (дата обращения: 28.01.2026). – Загл. с экрана.

28) Репозиторий курсовой работы «Веб-приложение для аренды автомобилей» [Электронный ресурс] – Режим доступа: <https://github.com/dimavalencko/carsharing-app> (дата обращения: 01.02.2026). – Загл. с экрана.